CHAPTER 6

Tokens, Tokenization and Embeddings

In this chapter

Introduction: The Invoice That Went Up Thirty Per Cent Overnight . . . 2
6.1 What a Tokenizer Actually Is . . . 3
6.2 Byte-Pair Encoding, From First Principles . . . 4
6.3 The Failure That Pre-tokenization Fixes . . . 6
6.4 Special Tokens and the Chat Template . . . 7
6.5 Why the Same Meaning Costs Different Amounts . . . 8
6.6 Counting Tokens Properly . . . 10
6.7 What an Embedding Actually Encodes . . . 12
6.8 Token Embeddings Versus Passage Embeddings . . . 14
6.9 Code Walkthrough: A Tokenizer in Ninety Lines . . . 17
6.10 Mini Project: A Tokenizer Built, Trained and Benchmarked . . . 19
6.11 Hands-on Labs . . . 22
6.12 Lab Solutions . . . 23
6.13 Best Practices . . . 24
6.14 Common Mistakes . . . 24
6.15 Performance Considerations . . . 26
6.16 Security Considerations . . . 26
6.17 Real Production Examples . . . 27
6.18 Certification Notes . . . 27
6.19 Practice Questions . . . 28
6.20 Answers and Explanations . . . 29
6.21 Ten Practical Exercises . . . 31
6.22 Five Challenge Labs . . . 31
6.23 Interview Questions . . . 32
6.24 Summary . . . 32
6.25 Glossary . . . 33
6.26 Further Reading . . . 34

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what a **tokenizer** is, what a **vocabulary** is, and why models read subword pieces rather than characters or words.
2. Describe **byte-pair encoding** precisely enough to implement it, and trace the merges it learns on a corpus you choose.
3. Explain why real tokenizers **pre-tokenize with a regular expression** before applying BPE, and describe the specific failure that occurs without it.
4. Account for why the **same meaning costs very different numbers of tokens** in different languages, and in code, JSON, UUIDs and base64.
5. Identify **special tokens** and explain how a chat template turns a list of messages into one flat sequence.
6. Count tokens correctly using the **token counting endpoint**, and explain why character-based estimates and third-party tokenizers are unsafe.
7. Explain what an **embedding vector** encodes, compute **cosine similarity**, and state the difference between a token embedding and a passage embedding.
8. Treat **chunking as a token-budgeting problem**, and quantify what overlap costs in storage and money.
9. Connect all of the above to context budgeting, multilingual pricing and prompt design decisions you will make in production.

NOTE

Prerequisites

Chapter 1 for tokens, the context window and the inference lifecycle — particularly section 1.1.7 and Figure 1.2. **Chapter 4** for the embedding and unembedding layers either end of the transformer stack, and for the parameter table in section 4.8. **Chapter 5** for embeddings as a retrieval artefact, cosine similarity and the two-stage retrieval pipeline of section 5.10.

Chapter 1 promised that you should "count tokens properly with the SDK rather than estimating". Chapter 5 closed by deferring the construction of the tokenizer to this chapter. Both debts are settled here.

Introduction: The Invoice That Went Up Thirty Per Cent Overnight

A team migrates a document-processing pipeline to a newer Claude model. Nothing else changes. Same prompts, same documents, same traffic, same volume of requests. The following month the invoice is roughly a third higher, and a batch of long documents that had always fitted comfortably now fails with a context-length error.

Nobody changed the prompt. Nobody changed the corpus. What changed was the **tokenizer**.

According to the official documentation at the time of writing, Claude 4.7 and later models use a newer tokenizer, and the same input text produces **approximately thirty per cent more tokens** than on earlier models. The documentation is explicit about the operational consequence: do not reuse token

counts measured against an earlier model, because usage and billing reflect the new tokenizer's counts.

This is not a bug and it is not a price rise. It is the single most under-appreciated fact about production language-model systems: **the token is the unit of cost, the unit of the context limit and the unit of latency, and the mapping from your text to tokens is a property of the model, not a property of your text.** Change the model and you change the arithmetic of your entire system.

That team's mistake was not migrating. It was holding a number in a spreadsheet — "our average document is 4,000 tokens" — that was never a fact about their documents. It was a fact about a tokenizer they had stopped thinking about.

This chapter makes the tokenizer stop being invisible. We will build one from scratch, train it on a real corpus, watch it learn merges, break it in an instructive way, fix it the way real tokenizers do, and then measure precisely what it costs to say the same sentence in ten languages. Then we turn to the other half of the story: what happens to a token id once it becomes a vector, why arithmetic on those vectors means anything at all, and why chunking a document is a budgeting exercise rather than a text-splitting exercise.

6.1 What a Tokenizer Actually Is

Start with the constraint. A neural network, as Chapter 3 established, consumes numbers. Specifically it consumes vectors of floating-point numbers. Text is not numbers. Something must sit between them, and that something is the tokenizer.

> DEFINITION
>
> **Tokenizer, vocabulary, token, token id**
>
> A **tokenizer** is a deterministic, reversible function that converts a string into a sequence of integers, and back again. It is not a neural network, it has no parameters that are trained by gradient descent, and it does not change during inference.
>
> A **vocabulary** is the fixed, finite set of pieces the tokenizer knows. A **token** is one member of that set — typically a fragment of a word, a whole common word, a punctuation mark or a run of whitespace. A **token id** is that token's index in the vocabulary: an integer.
>
> The vocabulary is chosen **before training begins** and cannot change afterwards, because the model's embedding matrix and output layer both have one row per vocabulary entry (Chapter 4, section 4.8). Changing the vocabulary means retraining the model.

Three designs are possible, and understanding why the middle one won explains everything that follows.

Character-level. A vocabulary of a few hundred characters. Nothing is ever unknown, and the vocabulary is tiny. But sequences become enormous — a 2,000-word document is roughly 12,000 characters — and since attention cost is quadratic in sequence length (section 4.10), this is ruinous. The character-level language model of section 3.10 was built this way precisely so you could feel the limitation.

Word-level. A vocabulary of whole words. Sequences are short, but the vocabulary must be huge, and it is still never big enough. Any word not in it — a surname, a product code, a typo, a newly coined term — becomes an **out-of-vocabulary** token, and the model is simply blind to it. This is fatal for code, for proper

nouns, and for every language with rich morphology.

Subword. A vocabulary of fragments. Common words get one token each; rare words are assembled from several pieces. The vocabulary stays manageable, sequences stay short, and — critically — **nothing is ever out of vocabulary**, because the pieces bottom out at individual bytes.

KEY TAKEAWAYS

Why subword tokenization won

Subword tokenization is the only one of the three that gets all of the following at once:

- **Short sequences**, which matter because attention cost grows quadratically with length.
- **A bounded vocabulary**, which matters because the embedding and unembedding matrices are `vocabulary x d` — at `d = 8192`, every 1,000 vocabulary entries is roughly 8 million parameters, twice over.
- **No out-of-vocabulary failures**, because a byte-level fallback can spell out anything, including text in a script the tokenizer has never seen.
- **Graceful degradation**, not catastrophic failure, on unfamiliar input. Unfamiliar text is not rejected; it is merely expensive.

That last property is the one to hold on to, because it is where the money goes. A byte-level subword tokenizer never fails on unfamiliar input. It just charges you more for it, silently, forever.

6.2 Byte-Pair Encoding, From First Principles

Byte-pair encoding began life in 1994 as a data-compression algorithm and was repurposed for tokenization in 2015. It is remarkably simple — you can hold the whole algorithm in your head — and it is the basis of the tokenizers used by essentially every major language model.

The idea in one sentence: **start with individual bytes, then repeatedly find the most frequent adjacent pair and glue it into a new single token, until you have as many tokens as you want.**

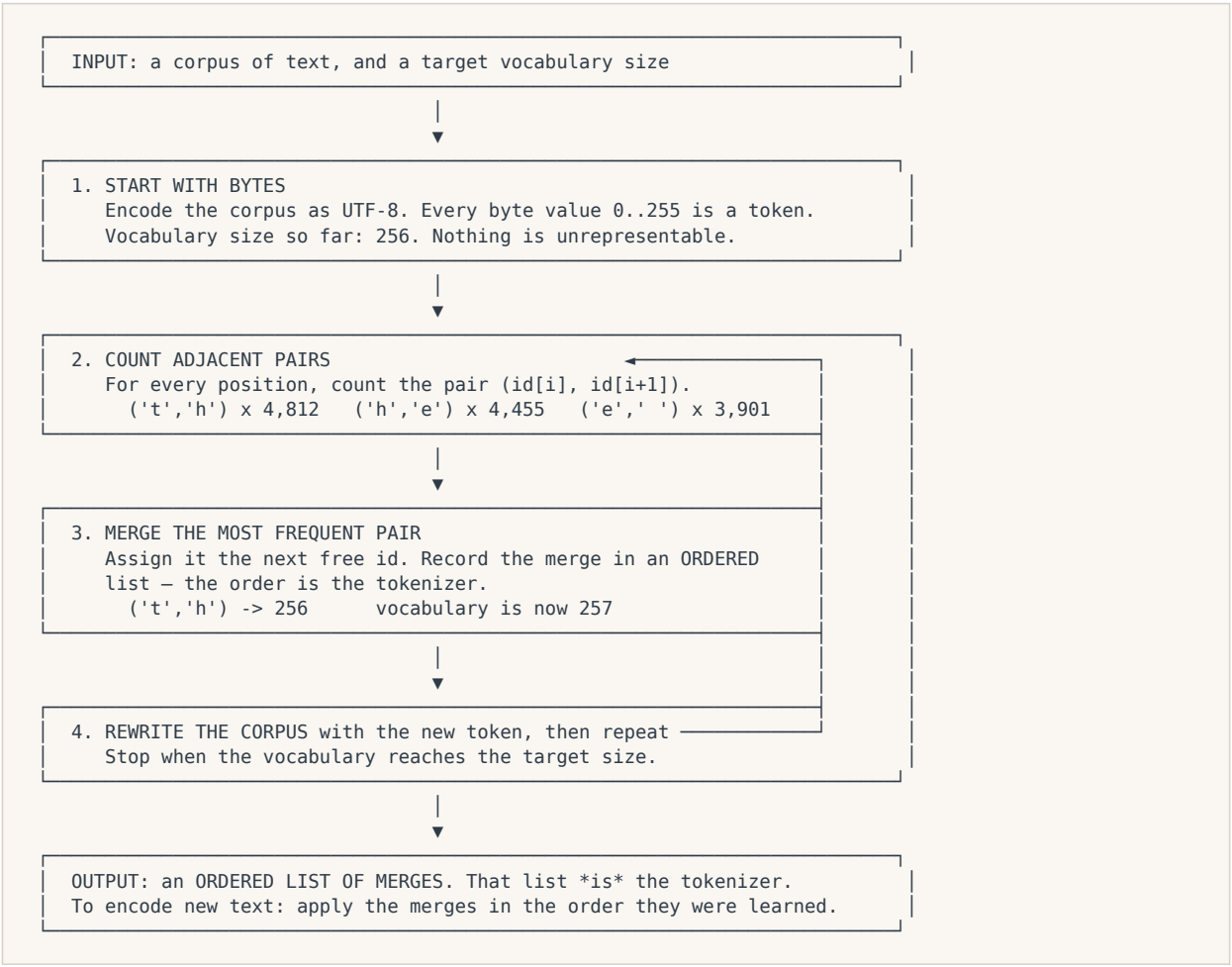


Figure 6.1 — The BPE training loop.

Two properties of this deserve emphasis, because both are examinable and both have production consequences.

The merge list is ordered, and the order is the algorithm. Encoding new text does not mean "find the longest matching vocabulary entry". It means "apply merge 1 wherever it applies, then merge 2, then merge 3". A different order produces a different tokenization of the same string. This is why a tokenizer is distributed as a merge list, not merely as a word list.

The tokenizer is trained, but not by gradient descent. It has a training corpus and it learns from statistics, but there is no loss function, no backpropagation and no learning rate. It is counting. This distinction matters when someone proposes "fine-tuning the tokenizer" — the concept does not apply. You can train a new tokenizer, but then you have a new vocabulary, and therefore you need a new model.

6.2.1 A worked example you can check by hand

Take the string `the cat sat on the mat, the cat sat flat.` and run four merges.

Step	Most frequent pair	Count	New token	Corpus after the merge
Start	—	—	—	`t h e ␣ c a t ␣ s a t ...` (40 byte tokens)
1	`a` + `t`	5	`at`	`t h e ␣ c at ␣ s at ...`

Step	Most frequent pair	Count	New token	Corpus after the merge
2	`at` + `␣`	3	`at␣`	`t h e ␣ c at␣ s at␣ ...`
3	`t` + `h`	3	`th`	`th e ␣ c at␣ s at␣ ...`
4	`th` + `e`	3	`the`	`the ␣ c at␣ s at␣ ...`

Notice what the algorithm has discovered without being told anything about English: that `at` is a common ending, that `the` is a common word, and — in step 2 — that a trailing space belongs with what precedes it. Nobody encoded a rule about spaces. Frequency found it.

Notice also that the merges compound. `the` at step 4 was only possible because `th` existed at step 3. Every token in a BPE vocabulary above 256 is a binary tree of earlier merges, bottoming out at bytes.

6.3 The Failure That Pre-tokenization Fixes

Now run that same naive algorithm for twenty merges instead of four, and something goes wrong. Here is the real output from the implementation we build in section 6.9, trained on that same forty-character sentence:

```
MiniBPE (20 merges learned)
  last 6 pieces: ['the cat sat on the mat',
                  'the cat sat on the mat,',
                  'the cat sat on the mat, ',
                  'the cat sat on the mat, the cat sat ',
                  'the cat sat on the mat, the cat sat f',
                  'the cat sat on the mat, the cat sat fl']
  encode('the cat sat') -> ['the cat s', 'at']
```

The tokenizer has learned to swallow the entire sentence into single tokens, and it segments `the cat sat` as `['the cat s', 'at']`. That is not a tokenizer. That is a lookup table for one sentence.

The failure is instructive rather than embarrassing, and it generalises. Pure frequency-driven BPE will happily merge across word boundaries, across the boundary between letters and digits, and across the boundary between text and punctuation. On a real corpus this produces tokens like `the same` and `2023-` and `);` — pieces that encode a specific collocation rather than a reusable unit of meaning. The vocabulary fills up with phrases that appear in the training corpus and are useless anywhere else, and generalisation collapses.

The fix, used by every production tokenizer, is **pre-tokenization**: split the text with a regular expression *first*, then run BPE independently inside each fragment. Merges can never cross a fragment boundary, because the fragments are never adjacent to one another during training.

DEFINITION

Pre-tokenization

Pre-tokenization is a deterministic split of the input string into chunks, applied before BPE and enforced during both training and encoding. Its purpose is to forbid merges that cross linguistically meaningless boundaries.

The GPT-2 pattern, which is the one most widely copied, splits on four rules: common English contractions (`'s`, `'ll`, `'ve`), runs of letters with an optional leading space, runs of digits with an optional leading space, and runs of punctuation with an optional leading space — with whitespace handled separately.

The same sentence, same twenty merges, with pre-tokenization switched on:

```
MiniBPEPretok (15 merges learned)
  last 6 pieces: [' on', ' m', ' mat', ' f', ' fl', ' flat']
  encode('the cat sat') -> ['the', ' cat', ' sat']
```

That is a tokenizer. Three words, three tokens, and the leading space belongs to the word that follows it — exactly the behaviour Chapter 1 described in section 1.1.7 without explaining where it came from. It came from here.

WARNING

The leading space is part of the token

`cat` and `cat` are **different tokens with different ids**. This is not a quirk; it is the direct result of pre-tokenization keeping a leading space with the following word.

The practical consequence appears constantly in prompt engineering. A trailing space at the end of your prompt changes which token the model must produce next, and can measurably degrade output — because you have forced it to continue with a token that never follows a space in training data. When you write a prompt that ends `Answer: `, you have made the model's job harder than if you had written `Answer:`. Chapter 20 returns to this.

6.3.1 Why the digit rule exists

Separating digits from letters is not cosmetic. Without it, a corpus containing `2023` frequently would produce a single token for `2023` and nothing useful for `2024`. Arithmetic, dates and version numbers would each be tokenized inconsistently depending on how fashionable they were in the training corpus. Splitting digits into their own runs makes numeric tokenization uniform, which is one of several reasons models handle arithmetic better than they used to.

6.4 Special Tokens and the Chat Template

Everything so far concerns ordinary text. But a conversation is not ordinary text. It is a structured object: a system prompt, then alternating user and assistant turns, possibly with tool calls and tool results interleaved. The model, however, consumes exactly one flat sequence of token ids.

DEFINITION

Special token

A **special token** is a vocabulary entry that does not correspond to any text a user can type. It is inserted by the serving layer to mark structure — the start of a document, the boundary between conversational turns, the point at which the model should stop.

Because they cannot be produced by tokenizing user text, special tokens are reserved: the tokenizer is configured never to emit them from ordinary input. This reservation is a **security boundary**, and section 6.16 explains what happens when it leaks.

The transformation from a structured request to a flat sequence is the **chat template**. Conceptually it looks like this:

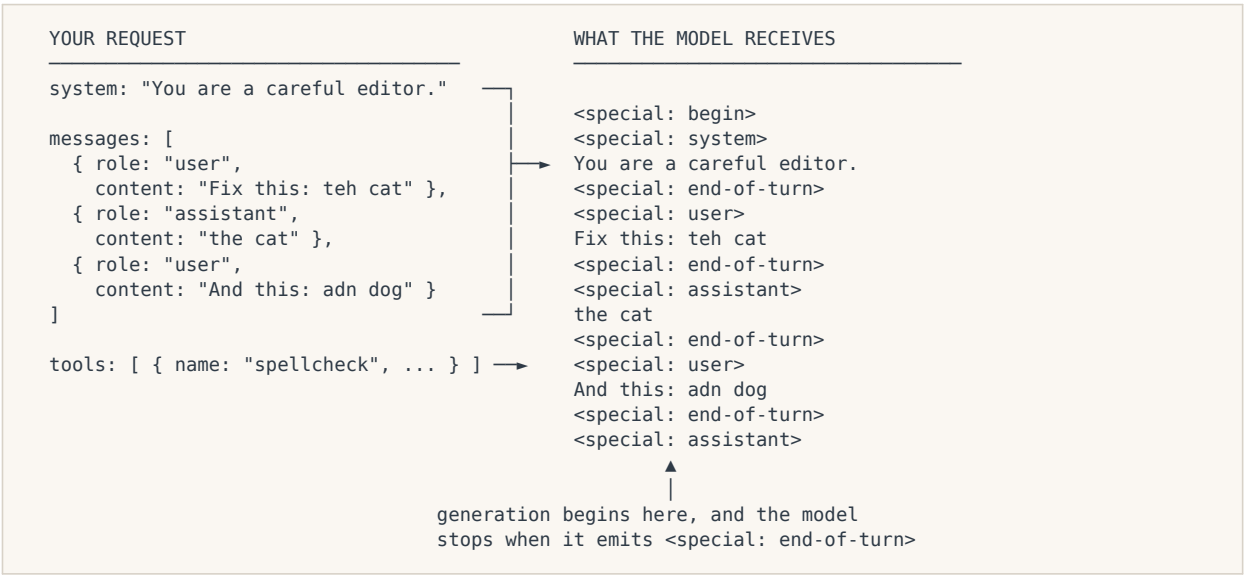


Figure 6.2 — From a structured request to one flat token sequence.

Three consequences follow directly, and all three are examinable.

The system prompt is not a message. As the running thread of this book puts it: the system prompt is a top-level parameter, not an entry in the `messages` array. Figure 6.2 shows why — it occupies a distinct structural slot in the flat sequence, marked by its own special token. Chapter 21 covers system prompt design in full.

Structure costs tokens. Every turn boundary is one or more tokens. A conversation of fifty short exchanges pays a structural overhead that a single long message does not. This is one reason the token counting endpoint returns a number larger than the sum of the message bodies.

`stop_reason` **is a branch on a special token.** When the model emits the end-of-turn token, generation stops and the API reports `stop_reason: "end_turn"`. When it hits your ceiling instead, you get `max_tokens`. Chapter 28 makes `stop_reason` the control-flow branch it deserves to be; the mechanism underneath it is a single token id.

6.5 Why the Same Meaning Costs Different Amounts

Here is where tokenization stops being a curiosity and starts appearing on invoices.

We trained the tokenizer of section 6.9 on 160 KB of this book's own English prose, to a vocabulary of 1,024. We then asked it to encode the same sentence — *"Artificial intelligence is changing how software is built"* — in ten languages. The measured results:

Language	chars	bytes	tokens	bytes/token	vs English
English	58	58	23	2.52	1.0x
French	77	77	38	2.03	1.7x
Spanish	78	78	42	1.86	1.8x
German	81	81	46	1.76	2.0x
Chinese	16	48	48	1.00	2.1x
Japanese	23	69	69	1.00	3.0x
Korean	30	78	78	1.00	3.4x
Arabic	43	80	80	1.00	3.5x
Russian	56	106	106	1.00	4.6x
Hindi	59	159	159	1.00	6.9x

Read the `bytes/token` column carefully, because it tells the whole story. For every non-Latin script it is exactly **1.00**. Not approximately — exactly. The tokenizer learned no merges at all for those byte sequences, so every single UTF-8 byte becomes its own token. Hindi is worst because Devanagari characters occupy three bytes each in UTF-8 and carry combining marks; 59 characters become 159 bytes become 159 tokens.

> WARNING
>
> **This measurement overstates the real effect, and here is by how much**
>
> Our tokenizer was trained exclusively on English technical prose, so it is the worst possible case. Production tokenizers are trained on deliberately multilingual corpora and learn merges for Cyrillic, CJK, Devanagari and Arabic script, so their non-English penalty is far smaller — typically in the range of 1.2x to 2.5x rather than 3x to 7x.
>
> The **direction** of the effect is real and universal; the **magnitude** here is inflated by our training data. Never quote our numbers as though they described a commercial model. Measure your own text against the model you will actually deploy, using the method in section 6.6.

The commercial consequence is direct and frequently overlooked. If you price a multilingual product on English measurements, your Hindi and Japanese users cost you substantially more per interaction than your English users — on input tokens, on output tokens, and on how much of the context window each conversation consumes before it must be truncated. A support assistant with a 200,000-token working budget has meaningfully less room for conversation history in Japanese than in English.

And the second-order effect is worse. The languages that tokenize badly are largely the same languages that have less training data, and therefore lower quality. The official multilingual benchmarks bear this out: at the time of writing, the documented zero-shot scores relative to English run at roughly 96–98%

for Spanish, French and German, around 96–97% for Japanese, Chinese and Hindi, but fall to about 91% for Swahili and around 80% for Yoruba on mid-tier models — and considerably lower on smaller ones. **You pay more per unit of meaning, and you get less of it.** Budget accordingly.

6.5.1 Content type matters as much as language

The same tokenizer, on six kinds of content:

Kind	bytes	tokens	bytes/token
English prose	79	21	3.76
Python code	52	27	1.93
Pretty-printed JSON	70	44	1.59
Minified JSON	52	38	1.37
UUID	36	35	1.03
Base64	44	43	1.02

Three lessons, each of which turns into money.

Structured data is expensive. JSON spends tokens on punctuation — braces, quotes, colons, commas — that carry structure rather than meaning. Chapter 23, on JSON contracts, weighs this against the reliability that structure buys; the point here is simply that the cost is real and measurable.

Pretty-printing is not free. In this sample the indented form cost 44 tokens against 38 for the minified form, on content that is semantically identical: a 16% premium for whitespace. On a high-volume pipeline embedding JSON into every prompt, that is a line item.

Random strings are pathological. A UUID costs roughly one token per character, because BPE learned no merges for arbitrary hex. Base64 is the same. If your prompts carry identifiers, hashes, embedded images as base64 or signed URLs, you are paying near the theoretical worst rate for every one of them. A common and highly effective optimisation is to replace long opaque identifiers with short reference keys and resolve them outside the model — a technique Chapter 60 develops as part of cost optimisation.

6.6 Counting Tokens Properly

Chapter 1 gave you the rule of thumb: roughly four characters per token, roughly 0.75 words per token, for ordinary English prose. Section 6.5 has just shown you how spectacularly that rule fails on Hindi, on base64 and on JSON. So how do you get a real number?

KEY TAKEAWAYS

Count, do not estimate

According to the official documentation at the time of writing, the Claude platform exposes a **token counting endpoint** at `POST /v1/messages/count_tokens`. It accepts the same structured input as the Messages API — system prompt, messages, tools, images and PDFs — and returns the input token count.

Four facts about it that matter operationally:

1. **It is free.** There is no charge for using it. It is, however, subject to its own requests-per-minute limits, documented at 2,000 RPM on the Start tier, 4,000 on Build and 8,000 on Scale.
2. **Its rate limits are independent** of the Messages API's. Using one does not consume the other's budget.
3. **Token counts are model-specific.** Pass the same model id you intend to use for inference. This is not a formality — it is the entire lesson of this chapter's introduction.
4. **The count is an estimate**, and the documentation says so: the actual number used when creating the message may differ by a small amount. Counts may also include tokens Anthropic adds for system optimisations, for which **you are not billed**.

The documented Python call, and its documented output:

```
import anthropic
client = anthropic.Anthropic()
response = client.messages.count_tokens(
    model="claude-opus-5",
    system="You are a scientist",
    messages=[{"role": "user", "content": "Hello, Claude"}],
)
print(response.json())
# {"input_tokens": 14}
```

Fourteen tokens for four words. The words themselves account for perhaps six; the rest is the chat template of Figure 6.2 — the structural special tokens. This is the overhead made visible, and it is why estimating from character counts underestimates short requests badly.

The documentation's other published figures are worth internalising as reference points: a request carrying a single small tool definition counts **403** input tokens, and one carrying a modest image counts **1,551**. Tool schemas and images are not free, and Chapter 32 returns to schema size as a cost lever.

WARNING

Do not use another vendor's tokenizer

A recurring production error is to count Claude tokens with `tiktoken`, which is OpenAI's tokenizer, because it is convenient and runs locally. Different model families have different vocabularies, so the counts are simply wrong — reported to undercount Claude tokens by roughly 15–20% on typical text.†

Undercounting is the dangerous direction. It produces budgets that are too small, prompts assembled just under a limit that turn out to be over it, and truncation that appears only in production on your longest and most valuable requests.

6.6.1 Migrating between models

Put the introduction's incident together with the tooling and you get the correct migration procedure. Because the token counting endpoint returns the count under the tokenizer of whichever `model` you pass, you can measure a change before you make it: count a representative sample of your real traffic twice — once with your current model id and once with the target — and compare the two `input_tokens` figures directly.

Do this on **your** content, not on a generic sample. The documented ~30% increase for the 4.7-generation tokenizer is an average; the actual figure depends on your content and workload shape. A pipeline dominated by base64 payloads will behave differently from one dominated by English prose.

6.7 What an Embedding Actually Encodes

A token id is an arbitrary integer. Token 4,891 is not "more" than token 12 in any sense — the numbering is an accident of merge order. So the first thing the model does with a token id is throw it away, by using it as a row index into a matrix.

> DEFINITION
>
> **Embedding (token sense)**
>
> A **token embedding** is the row of the embedding matrix corresponding to a token id: a vector of `d` floating-point numbers, where `d` is the model width. The matrix has one row per vocabulary entry, so its shape is `vocabulary x d` (Chapter 4, section 4.8).
>
> These vectors are **learned parameters**, adjusted by gradient descent during training like any other weight. Nobody specifies what they should contain. Their values are whatever made next-token prediction most accurate.

The remarkable empirical finding — the one that makes the word "embedding" worth caring about — is that these vectors arrange themselves geometrically by meaning. Words used in similar contexts end up pointing in similar directions. This was never a design goal. It is a consequence of the training objective: if two words can appear in the same slots, the cheapest way for the network to handle both is to give them similar representations.

6.7.1 Cosine similarity

Direction carries the meaning; length mostly carries frequency. So the standard comparison ignores length.

DEFINITION

Cosine similarity

Cosine similarity is the cosine of the angle between two vectors:

`cos(a, b) = (a · b) / (|a| x |b|)`

It ranges from −1 (opposite directions) through 0 (perpendicular, unrelated) to +1 (identical direction). Because it divides out both magnitudes, it compares *orientation only*.

If vectors are pre-normalised to unit length — which most embedding providers do, and which the Claude platform's recommended provider does — then `|a| = |b| = 1` and cosine similarity reduces to the plain dot product. This is why production retrieval code often computes a dot product and calls it cosine similarity: on normalised vectors they are the same number, and the dot product is faster.

To make this concrete we built 64-dimensional embeddings from the co-occurrence statistics of this book's first five chapters — roughly 47,000 words — using positive pointwise mutual information and a truncated SVD. This is a genuine, if old-fashioned, way to build embeddings, and it has the pedagogical virtue of being computable in a few seconds and inspectable line by line. The measured nearest neighbours:

```
NEAREST NEIGHBOURS BY COSINE SIMILARITY
  encoder    -> [('decoder', 0.841), ('bidirectional', 0.803),
                 ('cross-attention', 0.719), ('translation', 0.691),
                 ('embedding', 0.664), ('representations', 0.645)]
  gradient   -> [('descent', 0.857), ('along', 0.721), ('gradients', 0.681),
                 ('calculus', 0.671), ('slope', 0.667), ('downhill', 0.646)]
  cost       -> [('latency', 0.798), ('time-to-first-token', 0.700),
                 ('and', 0.664), ('merely', 0.664), ('quadratically', 0.659)]
  model      -> [('that', 0.744), ('an', 0.738), ('system', 0.691),
                 ('with', 0.689), ('not', 0.689), ('on', 0.688)]
```

The first three rows are exactly what we hoped for. `encoder` finds `decoder`, `bidirectional` and `cross-attention` — the vocabulary of Chapter 5, recovered from nothing but counting. `cost` finds `latency` and `time-to-first-token`, which is the thesis of Chapter 4's section 4.9 discovered statistically.

The fourth row is the honest failure, and it teaches more than the successes.

COMMON MISTAKE

Reading meaning into frequency artefacts

Symptom: `model` — arguably the single most important word in the corpus — has as its nearest neighbours `that`, `an`, `with` and `not`. Function words, carrying no meaning at all.

Cause: `model` appears so often, in so many different contexts, that its co-occurrence profile converges on the profile of the corpus as a whole. Its "context" is everything, so its vector is the average of everything — which is also where the function words sit.

Fix: frequency weighting. Down-weight very frequent terms, as the passage-embedding code in section 6.9 does with inverse-frequency weights. Trained neural embedding models handle this far better than raw co-occurrence, which is one concrete reason to use a purpose-built embedding model rather than rolling your own.

Detection: inspect nearest neighbours for your most frequent terms specifically. If the top matches are function words, frequency is dominating meaning.

VISUAL ANALOGY

A city map with no street names

Imagine a map of a city where nothing is labelled, but every building has been placed so that buildings serving similar purposes are near one another. Bakeries cluster in one district, law firms in another, hardware shops in a third. You could not read the map, but you could answer "what else is like this?" by drawing a circle. And you could answer "what is the equivalent of this bakery, over in the industrial quarter?" by measuring a direction and distance in one district and applying the same displacement in another.

That is an embedding space. Cosine similarity is the circle. Vector arithmetic is the displacement.

Where the analogy breaks: a city map is two-dimensional and the districts are contiguous and interpretable. An embedding space has hundreds or thousands of dimensions, individual dimensions almost never mean anything on their own, and meaning is carried by directions that are combinations of many dimensions at once. Worse, in high dimensions almost everything is roughly perpendicular to almost everything else, so raw cosine values cluster in a narrow band — which is why you should compare similarities against each other and rank them, never against an absolute threshold you picked by intuition.

6.7.2 Why arithmetic on embeddings works at all

The famous result is `king − man + woman ≈ queen`. It works — imperfectly, and less reliably than the folklore suggests — because the training objective has no way to encode "royalty" and "gender" other than as directions, and directions add.

If `king` and `queen` appear in near-identical contexts except for gendered pronouns, then the difference between their vectors is dominated by whatever the model uses to represent gender. The same difference separates `man` from `woman`. Subtracting one and adding the other therefore translates you along the gender direction while leaving the royalty direction untouched.

Two honest caveats, because this result is over-sold. First, the standard evaluation excludes the input words from the answer set; without that exclusion the nearest vector to `king − man + woman` is frequently `king` itself. Second, the effect is much weaker for rare words and for relations that are not cleanly linear. Treat vector arithmetic as evidence that the space has structure, not as a reliable operation to build a product on.

6.8 Token Embeddings Versus Passage Embeddings

Chapter 5 flagged a deliberate ambiguity in the word *embedding*, and now we can resolve it precisely, because the two things are produced by different machinery and used for different purposes.

	Token embedding	Passage embedding
What it represents	One vocabulary entry	An entire passage of text
Where it comes from	A row of the model's embedding matrix	The output of running a whole encoder over the text

	Token embedding	Passage embedding
When it is computed	Lookup — one array index	A full forward pass
Context-sensitive?	**No.** `bank` has one vector regardless of river or money	**Yes.** The whole passage informs it
What it is for	Feeding the first transformer block	Retrieval, clustering, classification
Typical dimension	The model width `d`	256 to 2,048, chosen by the provider

The context-sensitivity row is the important one. A token embedding is a **static** starting point; it is the transformer's attention layers, stacked, that make a representation context-dependent (Chapters 4 and 5). A passage embedding is the *output* of all that work, compressed to one vector.

KEY TAKEAWAYS

Anthropic does not offer an embedding model

According to the official documentation at the time of writing, **Anthropic does not offer its own embedding model**. The documentation names Voyage AI as an embeddings provider with a wide range of options, while explicitly encouraging you to assess multiple vendors for your use case.

Documented details worth knowing for architecture decisions: the current Voyage generation offers 32,000-token context with a default embedding dimension of 1,024 and selectable alternatives of 256, 512 and 2,048; there are code-, finance- and legal-specialised variants; `voyage-context` models produce contextualised chunk embeddings at up to 120,000-token context; rerankers are offered separately; and one model in the current generation is open-weight under Apache 2.0.

Two API details carry real architectural weight. Embeddings are **normalised to unit length**, so dot product and cosine similarity are the same number. And the API takes an `input_type` parameter distinguishing `"document"` from `"query"` — an **asymmetric** embedding scheme, where the same text embeds differently depending on which role it plays. Getting this backwards silently degrades retrieval quality with no error message anywhere.

To show a passage embedding working, we mean-pooled our token vectors with inverse-frequency weights and ran three queries against five documents. The measured result:

```
query : how does training actually update the weights
 -> +0.713  Gradient descent follows the slope of the loss downhill, ...
    +0.507  The tokenizer splits text into subword pieces drawn from ...
    +0.477  An encoder reads the whole input bidirectionally and prod...
```

Look closely at what happened. The query and the winning document share **no content words whatsoever** — not "training", not "update", not "weights", not "gradient", not "descent", not "loss". A lexical search of the kind that mis-ranked results in Chapter 5's section 5.10 would score this pair at zero and return nothing useful. The embedding scores it at 0.713 and ranks it first.

That is the entire argument for semantic retrieval, demonstrated on forty lines of NumPy.

6.8.1 Chunking is a token-budgeting problem

Documents are longer than embedding models accept and longer than you want to put in a context window, so they must be split. The instinct is to think of this as text processing — split on paragraphs, split on headings. That instinct produces systems that fail unpredictably, because the constraints are all measured in tokens and none of them are measured in paragraphs.

There are four budgets, and a chunk must satisfy all of them simultaneously: the embedding model's input limit; the context you can afford to spend at generation time, which is `chunk_size` x `number_of_chunks_retrieved`; the granularity that makes retrieval precise, since a chunk containing five topics matches all five weakly; and the coherence that makes a chunk answerable on its own.

Overlap is the standard mitigation for the fact that a fixed boundary will sometimes cut an answer in half. It is not free, and the cost is exactly computable. Measured on our tokenizer:

Budget	Overlap	Chunks	Tokens stored	Storage multiplier
128	0	17	2,097	1.00x
128	24	21	2,570	1.23x
512	64	5	2,353	1.12x

A 24-token overlap on 128-token chunks costs 23% more storage, 23% more vectors to index, 23% more embedding API calls to build the index and 23% more of everything to keep it current. That may well be worth it. The point is that it is a decision with a number attached, and you should know the number. Chapter 53 develops chunking strategy in full; what belongs here is the discipline of measuring it in tokens.

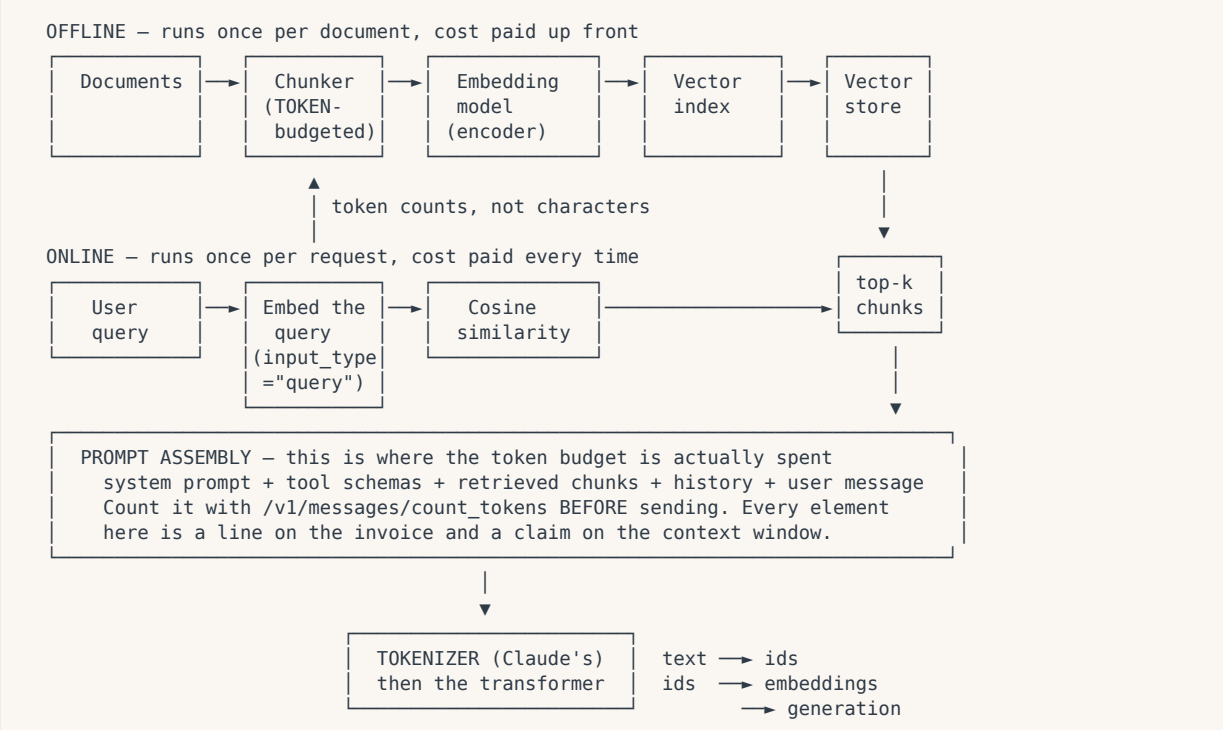


Figure 6.3 — Where tokenization and embeddings sit in a real system.

Notice that two entirely different tokenizers appear in this diagram: the one inside your chunker, which should be the embedding provider's, and Claude's own, which governs what you are billed for the

assembled prompt. They are not the same and they do not agree. Chunk to the embedding model's budget; bill against Claude's count.

6.9 Code Walkthrough: A Tokenizer in Ninety Lines

Everything in sections 6.2 and 6.3 is implemented here. The whole tokenizer is two functions and one class, and it is worth reading in full because there is nothing hidden in it.

The two primitives first. `get_stats` counts adjacent pairs; `merge` rewrites a sequence, replacing one pair with one new id.

```
from collections import Counter
def get_stats(ids, counts=None):
    """Count every adjacent pair in a sequence of token ids."""
    counts = Counter() if counts is None else counts
    for pair in zip(ids, ids[1:]):
        counts[pair] += 1
    return counts
def merge(ids, pair, new_id):
    """Replace every occurrence of `pair` with the single id `new_id`."""
    out, i = [], 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] == pair[1]:
            out.append(new_id)
            i += 2
        else:
            out.append(ids[i])
            i += 1
    return out
```

- `zip(ids, ids[1:])` is the idiomatic way to walk adjacent pairs. For `[1,2,3]` it yields `(1,2)` then `(2,3)`.
- `counts` **is passed in optionally** so the caller can accumulate statistics across many chunks into one `Counter` — which is exactly what training does once pre-tokenization splits the corpus.
- `i += 2` **on a match** is the subtle line. Having consumed two ids, we must skip both. Getting this wrong produces overlapping merges and a tokenizer that silently corrupts text.

Now the class. Training is Figure 6.1 transcribed directly.

```
class MiniBPE:
    """Naive byte-level BPE: no pre-tokenization."""
    def __init__(self):
        self.merges = {}
        self.vocab = {i: bytes([i]) for i in range(256)}
    def _chunks(self, text):
        return [text]
    def train(self, text, vocab_size, verbose_every=0):
        assert vocab_size >= 256
        chunks = [list(c.encode("utf-8")) for c in self._chunks(text)]
        for k in range(vocab_size - 256):
            stats = Counter()
            for ch in chunks:
                get_stats(ch, stats)
            if not stats:
                break
```

```
            pair = max(stats, key=stats.get)
            new_id = 256 + k
            chunks = [merge(ch, pair, new_id) for ch in chunks]
            self.merges[pair] = new_id
            self.vocab[new_id] = self.vocab[pair[0]] + self.vocab[pair[1]]
        return self
```

- `self.vocab` **starts with all 256 byte values.** This is what makes the tokenizer total: any byte sequence whatsoever can be represented, so there is no such thing as out-of-vocabulary input.
- `text.encode("utf-8")` is where characters become bytes. Everything downstream is integers in `0..255` plus learned ids above 255.
- `assert vocab_size >= 256` guards the one input that makes no sense. You cannot have a vocabulary smaller than the byte alphabet you started from.
- `if not stats: break` handles a corpus so small it runs out of pairs to merge. Our forty-character demo hits this: `MiniBPEPretok` learns only 15 of the 20 merges requested, because pre-tokenization leaves it nothing further to merge. An honest early stop rather than a crash.
- `max(stats, key=stats.get)` picks the most frequent pair. Ties are broken by insertion order, which is deterministic in Python but arbitrary in meaning; production tokenizers usually break ties explicitly for reproducibility.
- `self.merges[pair] = new_id` records the merge. In Python 3.7+ dictionaries preserve insertion order, so `self.merges` *is* the ordered merge list, and the ids themselves encode the order.
- `self.vocab[new_id] = vocab[a] + vocab[b]` builds the human-readable byte string for the new token by concatenating its parents. This is the binary tree of section 6.2 being constructed bottom-up.

Encoding applies the merges in learned order:

```
    def _encode_chunk(self, chunk):
        ids = list(chunk.encode("utf-8"))
        while len(ids) >= 2:
            stats = get_stats(ids)
            pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
            if pair not in self.merges:
                break
            ids = merge(ids, pair, self.merges[pair])
        return ids
    def encode(self, text):
        out, cache = [], {}
        for chunk in self._chunks(text):
            if chunk not in cache:
                cache[chunk] = self._encode_chunk(chunk)
            out.extend(cache[chunk])
        return out
```

- `min(..., key=lambda p: self.merges.get(p, inf))` is the heart of encoding, and it repays careful reading. Among all pairs currently present, we want the one that was learned *earliest*. Since ids increase with merge order, the lowest id is the earliest merge. Pairs never merged return infinity and therefore lose.
- **If the winning pair is not in** `merges`**, no merge applies at all** and we stop. That is the termination condition.

- **The `cache` in `encode`** is a large practical win. Once pre-tokenization is on, chunks are words, and words repeat constantly. Encoding our 160 KB corpus without this cache took minutes; with it, seconds.

Decoding is trivial, which is the point — the vocabulary already holds the byte string for every id:

```
    def decode(self, ids):
        raw = b"".join(self.vocab[i] for i in ids)
        return raw.decode("utf-8", errors="replace")
```

- `errors="replace"` matters more than it looks. A sequence of ids may slice through the middle of a multi-byte UTF-8 character — entirely possible if a model generates a partial character at a `max_tokens` boundary. Without this argument, decoding raises; with it, you get the replacement character. This is precisely why streaming clients sometimes show a brief `?` that resolves on the next chunk, a phenomenon Chapter 17 returns to.

Finally, the subclass that fixes section 6.3's failure. It is six lines.

```
import regex as re
GPT2_PAT = (r"'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+"
            r"| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+")
class MiniBPEPretok(MiniBPE):
    """Byte-level BPE with GPT-2 style regex pre-tokenization."""
    def __init__(self):
        super().__init__()
        self._pat = re.compile(GPT2_PAT)
    def _chunks(self, text):
        return self._pat.findall(text)
```

- `import regex`**, not** `re`**.** The standard library's `re` does not support Unicode property classes like `\p{L}`. This is not incidental: `\p{L}` means "any letter in any script", which is what makes the pattern work on Japanese and Cyrillic rather than only on ASCII.
- `?\p{L}+` is the rule that attaches a leading space to the following word, and therefore the direct cause of `cat` and `cat` being different tokens.
- `\s+(?!\S)` captures runs of whitespace not followed by a non-space character — trailing whitespace and blank lines — so that indentation in code becomes its own token rather than being glued onto an identifier.
- **The entire fix is overriding `_chunks`.** Training and encoding are unchanged; the base class was written with that seam deliberately in place.

6.10 Mini Project: A Tokenizer Built, Trained and Benchmarked

Goal. Build a working byte-level BPE tokenizer as an installable package, train it on a real corpus, verify it against a production tokenizer library on identical data, and produce a cost report that turns token counts into money.

Architecture.

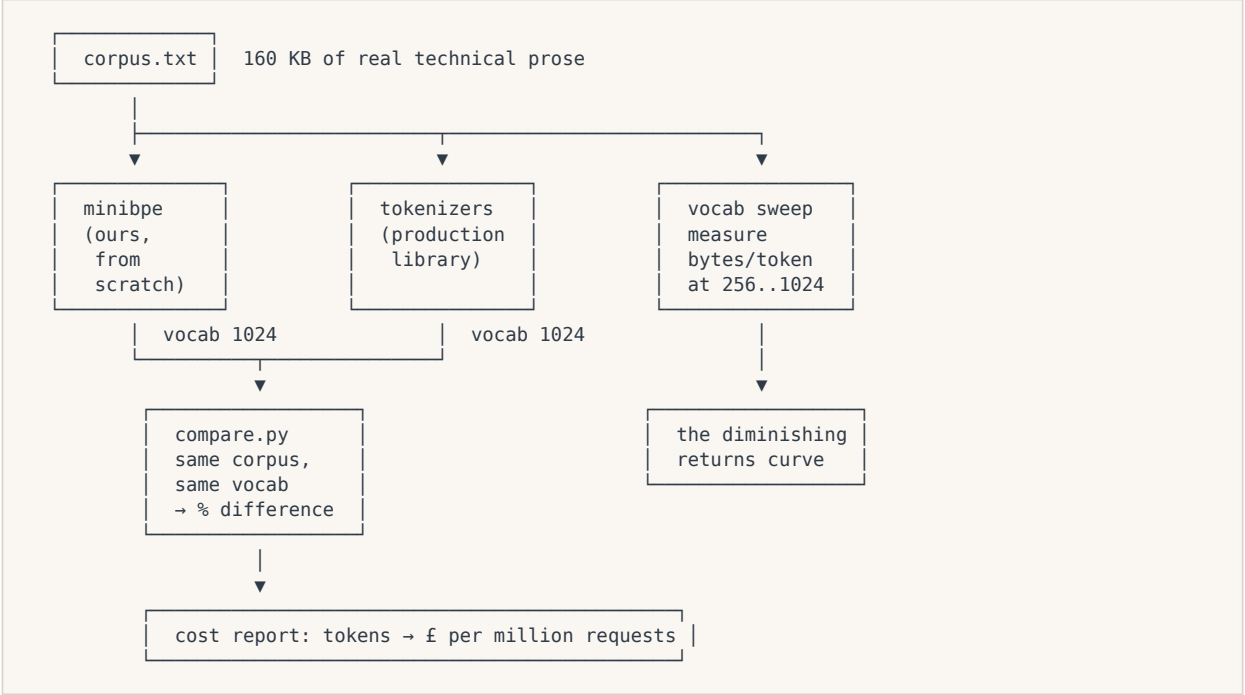


Figure 6.4 — The mini project.

Folder structure.

```
tokenlab/
├── minibpe.py          the tokenizer (section 6.9, complete)
├── build_corpus.py     assemble corpus.txt from source documents
├── train_minibpe.py    train ours to vocab 1024, save minibpe.json
├── train_hf.py         train the production library, same corpus/vocab
├── compare.py          benchmark ours against theirs
├── langs.py            language and content-type cost table
├── vocab_curve.py      bytes/token as a function of vocabulary size
├── test_minibpe.py     the test suite
└── corpus.txt          the training data
```

Training our tokenizer. Real measured output:

```
training byte-level BPE (with pre-tokenization)
corpus: 160,381 bytes, target vocab_size=1024
  merge  256: (282, 280) -> 511  ' wor'  (count 68)
  merge  512: (341, 346) -> 767  ' exam'  (count 31)
  merge  768: (267, 268) -> 1023 ' then' (count 19)
trained in            : 33.1s
merges learned        : 768
bytes in              : 160,381
tokens out            : 54,123
bytes per token       : 2.96
round-trip            : OK
```

Read the merge trace as a story about diminishing returns. Merge 256 captures `wor` 68 times; merge 768 captures `then` only 19 times. The algorithm spends its early merges on genuinely common English fragments and its late merges on increasingly marginal ones.

Benchmarking against a production library. We train the Hugging Face `tokenizers` library on the identical corpus at the identical vocabulary size, so the comparison isolates implementation quality

rather than confounding it with corpus or vocabulary differences. Real measured output:

```
Whole corpus, both at vocab_size = 1024
  bytes                : 160,381
  minibpe tokens       : 54,123   (2.96 bytes/token)
  tokenizers tokens    : 54,425   (2.95 bytes/token)
  difference           : -0.6%
Same sentence, token by token
  text     : 'The architect must count tokens, not estimate them.'
  minibpe  : ['The', ' architect', ' m', 'ust', ' c', 'oun', 't',
              ' tokens', ',', ' not', ' ', 'est', 'im', 'ate', ' them', '.']
  tokenizer: ['The', 'Ġarchitect', 'Ġm', 'ust', 'Ġc', 'oun', 't',
              'Ġtokens', ',', 'Ġnot', 'Ġ', 'est', 'im', 'ate', 'Ġthem', '.']
Round-trip on held-out text
  minibpe  : True
  tokenizer: True
```

Ninety lines of Python land within **0.6%** of a heavily optimised Rust implementation, and the segmentations are token-for-token identical. The `Ġ` is simply that library's printable stand-in for a leading space — the same our implementation shows. BPE is genuinely this simple; the production libraries are faster, not cleverer.

The segmentation itself is worth studying. `architect` and `tokens` each became a single token because they appear often in this corpus. `must` split into `m` + `ust`, and `estimate` into `est` + `im` + `ate`, because at a 1,024-entry vocabulary there was no room for them. Vocabulary size is exactly the dial that decides which words are "common enough" to be cheap.

The vocabulary-size curve. Measured on the same corpus, instrumenting a single training run:

Vocabulary	Tokens	Bytes/token	Tokens as % of bytes
256	160,381	1.00	100.0%
320	98,330	1.63	61.3%
384	84,216	1.90	52.5%
512	71,706	2.24	44.7%
768	60,358	2.66	37.6%
1,024	54,123	2.96	33.7%

The first 64 merges cut the sequence by 39%. The next 704 merges cut it by only a further 28 percentage points. This is the fundamental trade-off in vocabulary sizing: every doubling of the vocabulary buys less compression than the last, while costing `d` parameters per entry in the embedding matrix *and* `d` per entry in the unembedding matrix. Production vocabularies in the range of 100,000 to 200,000 entries sit where those two curves cross for multilingual corpora.

Tests. A tokenizer that silently corrupts text is worse than no tokenizer, so the round-trip property is the test that matters most.

```
import random
import string
from minibpe import MiniBPEPretok, get_stats, merge
```

```
def test_get_stats_counts_adjacent_pairs():
    assert get_stats([1, 2, 1, 2, 3]) == {(1, 2): 2, (2, 1): 1, (2, 3): 1}
def test_merge_replaces_and_skips():
    assert merge([1, 2, 1, 2], (1, 2), 9) == [9, 9]
    assert merge([1, 1, 1], (1, 1), 9) == [9, 1]
def test_roundtrip_on_training_text():
    text = open("corpus.txt", encoding="utf-8").read()[:20000]
    tok = MiniBPEPretok().train(text, 400)
    assert tok.decode(tok.encode(text)) == text
def test_roundtrip_on_unseen_scripts():
    tok = MiniBPEPretok().load("minibpe.json")
    for s in ["□□□□□□□□", "Здравствуй, мир", "□□", "",
              "a" * 500, "\t\n  mixed \r\n whitespace  "]:
        assert tok.decode(tok.encode(s)) == s
def test_roundtrip_on_random_bytes():
    tok = MiniBPEPretok().load("minibpe.json")
    alphabet = string.printable
    for _ in range(200):
        s = "".join(random.choice(alphabet) for _ in range(50))
        assert tok.decode(tok.encode(s)) == s
def test_leading_space_is_a_different_token():
    tok = MiniBPEPretok().load("minibpe.json")
    assert tok.encode("cat") != tok.encode(" cat")
```

Real output:

```
$ python3 -m pytest test_minibpe.py -q
......                                                   [100%]
6 passed in 1.40s
```

`test_roundtrip_on_unseen_scripts` is the one that earns its place. Japanese, Cyrillic, emoji, the empty string, a long repeated run and mixed whitespace all round-trip through a tokenizer trained exclusively on English — because byte-level BPE is total. Emoji are worth singling out: they are four-byte UTF-8 sequences, so they exercise the multi-byte path that `errors="replace"` exists to protect.

Deployment. A tokenizer is a build-time artefact, not a service. Ship `minibpe.json` alongside your application, load it once at start-up, and treat it as immutable and version-pinned. If you ever regenerate it, every embedding and every cached count computed under the old one is invalid. Version the file, log the version with every count you record, and never overwrite in place.

Evaluation. Judge a tokenizer on four measurements, in this order: **round-trip fidelity** on adversarial input, which must be 100% and is a correctness bug otherwise; **bytes per token** on text representative of your real traffic, not on a generic corpus; **the fairness gap**, meaning bytes per token in your worst-served language divided by your best, which is a cost-equity number your product owner should see; and **encode throughput**, which matters only if you tokenize at request time rather than at build time.

6.11 Hands-on Labs

Lab 6.1 — Watch the merges happen. Train `MiniBPE` (the naive one) on a paragraph of your own writing with `vocab_size=300` and print every merge with its count. *Success criterion:* you can point to the first merge that crosses a word boundary and explain why frequency alone produced it.

Lab 6.2 — Reproduce the pre-tokenization fix. Train both `MiniBPE` and `MiniBPEPretok` on the same short text and compare the final six vocabulary entries. *Success criterion:* you can state the rule that prevents the naive failure, and identify which part of `GPT2_PAT` enforces it.

Lab 6.3 — Price your own corpus. Take 2,000 words of text your organisation actually processes. Measure bytes per token with a trained tokenizer, then compute the monthly token cost at 10,000 requests per day. *Success criterion:* a single number in pounds or dollars, plus the arithmetic that produced it.

Lab 6.4 — Find your worst-served language. Encode the same sentence in every language your product supports and rank by tokens. *Success criterion:* a fairness gap ratio, and a sentence stating what it means for your per-user margin.

Lab 6.5 — The overlap bill. Chunk a 50,000-token document at budgets of 256, 512 and 1,024 tokens with overlaps of 0%, 10% and 20%. *Success criterion:* a nine-cell table of storage multipliers, and a recommendation with a stated reason.

6.12 Lab Solutions

Solution 6.1. On ordinary English the first boundary-crossing merge is usually something like `e` + or + `t`, appearing within the first dozen merges. The reason is that the space character is the single most common byte in English text, so pairs involving it dominate the frequency counts immediately. Pure frequency has no concept of a word; it sees only that this pair occurs often. The lesson generalises: **an unconstrained statistical objective will optimise exactly what you measured, not what you meant** — a theme that returns in Chapter 7 when we look at reward hacking in RLHF.

Solution 6.2. The rule is that BPE runs independently inside each pre-tokenized chunk, so pairs spanning a chunk boundary are never counted and therefore never merged. In `GPT2_PAT` the clause `?\p{L}+` is the one doing the work: it matches an optional single leading space followed by letters *only*, so a chunk can never contain a space in its interior. `the cat` is therefore always at least two chunks, and `'the cat s'` is unreachable.

Solution 6.3. The arithmetic, using our measured 2.96 bytes per token as an illustration and a hypothetical rate:

```
2,000 words ≈ 12,000 bytes ≈ 12,000 / 2.96 ≈ 4,054 input tokens
10,000 requests/day × 30 days = 300,000 requests/month
300,000 × 4,054 = 1.216 billion input tokens/month
```

The instructive part is not the total. It is that improving bytes per token from 2.96 to 3.50 — achievable by moving from a 1,024-entry vocabulary to a production one — cuts that figure by about 15% with no change to your prompt whatsoever. And that a single unnecessary base64 blob in the prompt, at 1.02 bytes per token, can undo the entire saving.

Solution 6.4. Using our English-trained tokenizer the gap between English and Hindi is 6.9x. With a production multilingual tokenizer expect something closer to 1.5–2.5x. Either way the interpretation is the same: if you charge all users the same price and your costs scale with tokens, your Hindi users have a materially worse unit margin than your English users. The remedies are real and worth naming — shorter system prompts for high-cost languages, more aggressive history truncation, tighter `max_tokens`,

or explicit regional pricing.

Solution 6.5. Storage multiplier is `1 / (1 – overlap_fraction)`, so 10% overlap costs about 1.11x and 20% about 1.25x, independent of chunk size. Chunk size does not change the multiplier, but it does change everything else: small chunks retrieve precisely but fragment answers, large chunks preserve context but dilute relevance and cost more context per retrieved item. The defensible recommendation is the largest chunk that still holds a single coherent idea, with the smallest overlap that stops cutting your actual answers in half — and both of those are empirical questions you settle with the recall@k measurement of Chapter 5, not by intuition.

6.13 Best Practices

BEST PRACTICE

Ten rules that follow from this chapter

1. **Count tokens; never estimate them.** Use `/v1/messages/count_tokens`. It is free, and it is the only number that matches your invoice.
2. **Always pass the model id you will actually use.** Counts are model-specific, and the tokenizer changed between generations.
3. **Re-measure on every model migration.** Documented figures are averages; your content is not average.
4. **Never use another vendor's tokenizer to count Claude tokens.** It undercounts, and undercounting is the dangerous direction.
5. **Measure bytes per token per language** for every language you support, and treat the ratio as a cost-equity metric.
6. **Keep opaque identifiers out of prompts.** UUIDs, hashes and base64 cost roughly one token per character. Use short reference keys and resolve them outside the model.
7. **Minify JSON that is going into a prompt** unless a human needs to read it. Indentation is not free.
8. **Never end a prompt with a trailing space.** The leading space belongs to the next token, and you have just made that token unreachable.
9. **Chunk to a token budget, not a character or paragraph budget.** Use the embedding provider's tokenizer for chunking and Claude's for billing.
10. **Version-pin the tokenizer artefact and log its version with every stored count.** Regenerating it invalidates every count and every embedding you have.

6.14 Common Mistakes

COMMON MISTAKE

Estimating tokens from character counts

Symptom: prompts that fit in testing and fail in production with a context-length error, usually on the longest and most valuable requests. **Cause:** the four-characters-per-token rule applied to content that is not English prose. **Fix:** call the token counting endpoint. **Detection:** compare your estimate to the real count on your ten longest real requests; if the gap exceeds a few per cent, your estimator is unsafe.

COMMON MISTAKE

Reusing token counts across a model migration

Symptom: the invoice rises sharply with no change in traffic, and long inputs begin to overflow. **Cause:** treating a token count as a property of the text rather than of the model. **Fix:** re-count a representative sample against the target model before migrating. **Detection:** count the same request under both model ids and compare `input_tokens`.

COMMON MISTAKE

Counting Claude tokens with tiktoken

Symptom: budgets that are consistently slightly too small; occasional unexplained truncation. **Cause:** using OpenAI's tokenizer against an Anthropic model because it runs locally. **Fix:** the token counting endpoint. **Detection:** any import of `tiktoken` in a codebase that calls the Messages API is this bug.

COMMON MISTAKE

Pricing a multilingual product on English measurements

Symptom: unit economics that degrade as international adoption grows, with no obvious cause. **Cause:** the same meaning costs more tokens in most non-English languages. **Fix:** measure per language and model the blended rate against your actual traffic mix. **Detection:** plot cost per conversation segmented by user locale.

COMMON MISTAKE

Chunking by characters or paragraphs

Symptom: embedding calls rejected for over-length input, or retrieved context that unpredictably blows the prompt budget. **Cause:** every real constraint is measured in tokens; paragraphs vary in length by an order of magnitude. **Fix:** chunk with the embedding model's tokenizer to an explicit token budget. **Detection:** measure the token length distribution of your chunks; a long tail means you are not budgeting.

COMMON MISTAKE

Swapping the embedding model without re-indexing

Symptom: retrieval quality collapses immediately after a deployment that "only changed a model name". **Cause:** vectors from different models occupy incomparable spaces; cosine similarity between them is meaningless noise. **Fix:** re-embed the entire corpus, and treat the index as versioned by model. **Detection:** store the embedding model id with every vector and refuse queries where it does not match.

COMMON MISTAKE

Ignoring the input_type parameter

Symptom: retrieval that works but is quietly mediocre, with no error anywhere. **Cause:** embedding queries as documents, in a scheme designed to be asymmetric. **Fix:** pass `input_type="query"` for queries and `"document"` for corpus text. **Detection:** re-run your recall@k measurement with the parameter corrected; if it improves, this was the bug.

6.15 Performance Considerations

Tokenization itself is fast and rarely the bottleneck. A production tokenizer processes megabytes per second. The cost of tokenization is not the CPU time; it is the number of tokens it produces, which then flows into a quadratic attention cost (section 4.10) and a per-token price.

Prefill scales with input tokens; decode scales with output tokens. Chapter 4's section 4.9 established the asymmetry. Tokenization determines both numbers, so a 30% increase in tokens per byte is simultaneously a 30% increase in input cost, a proportional increase in time-to-first-token, and a 30% larger claim on the context window.

Fewer tokens means less attention dilution. Beyond cost, every token competes with every other for the model's attention. Reducing token count by removing base64 blobs and minifying JSON improves quality as well as price — the same insight Chapter 55 develops as attention dilution.

Prompt caching is measured in tokens and is prefix-only. The minimum cacheable prompt on Claude Opus 5 is **512 tokens**, reduced from 1,024 on Opus 4.8. Whether your system prompt clears that threshold is a tokenization question, and the answer changes when the tokenizer changes. Chapter 30 covers caching economics in full.

Embedding is an offline cost; retrieval is an online one. Embed once per document, compare vectors per query. A 1,024-dimensional float32 vector is 4 KB, so a million chunks is about 4 GB before index overhead — which is why the selectable 256-dimension option exists, and why Matryoshka-style truncation is worth understanding before you provision hardware.

6.16 Security Considerations

Special tokens are a trust boundary. If user-supplied text can be tokenized into the special tokens of Figure 6.2, a user can forge a turn boundary and impersonate the system prompt or a tool result. Well-built serving layers reserve special tokens so they cannot be produced from ordinary input. Never build your own chat template by string concatenation over untrusted input; use the API's structured `messages` array, which is precisely what it is for.

Token counts leak information. The length of a response, visible through streaming timing or billing records, correlates with its content. In a system where the mere existence of a record is confidential — a medical or legal lookup — an observer who can measure token counts can infer whether a query matched anything. Where this matters, pad responses or batch them.

Embeddings are confidential data. A vector is a lossy but far from meaningless representation of its source text, and published inversion techniques recover meaningful content from embeddings. A vector store built over confidential documents is a store of confidential data and needs the same access controls, encryption and retention policy as the documents themselves. Chapter 5 made this point; it bears repeating because vector databases are routinely provisioned with weaker controls than the document stores they index.

Unicode is an attack surface. Homoglyphs, zero-width characters, right-to-left overrides and unusual normalisation forms all tokenize differently from the text they visually resemble. An instruction that

looks identical to a human can be a different token sequence entirely. Normalise input to a canonical form before both filtering and tokenizing, and be aware that a filter operating on characters while the model operates on tokens is a filter with a gap in it.

Token-count limits are a denial-of-service control. Because cost is linear in tokens and attention is quadratic in sequence length, an attacker who can submit arbitrary long input can impose disproportionate cost. Enforce a maximum token count on untrusted input using a real count, not a character-length proxy — a character limit is trivially evaded with content that tokenizes badly on purpose.

6.17 Real Production Examples

Multilingual customer support. A support assistant measures cost per resolved conversation by locale and discovers that its Japanese and Hindi conversations cost roughly twice its English ones. The response is not to drop those languages but to shorten the system prompt, cache it aggressively, and truncate history more eagerly where tokens are expensive.

Document ingestion pipelines. A contract-analysis system chunks to a strict token budget measured with the embedding provider's tokenizer, stores the chunk token count alongside each vector, and assembles prompts by filling a token budget rather than by taking a fixed top-k. The result is predictable cost per request regardless of how long the retrieved chunks happen to be.

Log and telemetry analysis. Raw logs are full of UUIDs, timestamps and stack traces that tokenize at close to one token per character. Pipelines that strip or substitute identifiers before the prompt routinely see token counts fall by half, with no loss of analytical quality — because the model was never going to reason about a UUID anyway.

Model migration in a regulated environment. A financial services team migrating between model generations runs its entire month of archived traffic through the token counting endpoint against both model ids before switching, producing a documented cost delta and a list of requests that would newly exceed the context limit. That list, not the average, is what determines whether the migration is safe.

6.18 Certification Notes

Concept	Where it matters on the exam
Tokens as the unit of cost, limit and latency	Domain 5: context management; Domain 1: capacity and cost planning
Counting tokens with the endpoint rather than estimating	Domain 5: reliability; Domain 4: prompt sizing
Token counts are model-specific	Domain 1: model selection and migration risk
Language and content-type variation in token cost	Domain 5: budgeting for multilingual and structured payloads
Special tokens and the chat template	Domain 4: system prompts as a top-level parameter; Domain 2: tool result framing

Concept	Where it matters on the exam
Chunking as a token budget	Domain 5: retrieval design; Domain 1: RAG architecture
Embeddings, cosine similarity and `input_type`	Domain 5: retrieval quality diagnosis
Re-indexing when the embedding model changes	Domain 5: operational correctness of a vector store

CERTIFICATION NOTE

Exam tips carried forward

- When a scenario reports a cost increase after a **model change** with no traffic change, the answer involves **tokenization**, not pricing.
- When an option proposes estimating tokens from character or word counts, it is wrong. The correct option calls the counting endpoint.
- When an option proposes counting Claude tokens with a third-party tokenizer such as `tiktoken`, it is wrong.
- Anthropic **does not provide an embedding model**. Options claiming an Anthropic embeddings endpoint are wrong; the documented route is a third-party provider.
- When an embedding model changes, the corpus must be **re-embedded**. Options that keep the existing index are wrong.
- Chunking questions are token-budget questions. Options phrased in characters or paragraphs are usually the distractors.
- Remember that the **system prompt is a top-level parameter**, not a message — Figure 6.2 is the mechanical reason.

6.19 Practice Questions

1. A tokenizer's vocabulary is fixed at what point? (a) At the first API call. (b) Before model training begins. (c) Per request. (d) When the context window fills.

2. Byte-level BPE guarantees that: (a) All text costs the same. (b) No input is ever out-of-vocabulary. (c) Tokens equal words. (d) Counts match across vendors.

3. In BPE training, the pair chosen for each merge is the one that is: (a) Longest. (b) Most frequent. (c) Alphabetically first. (d) Rarest.

4. What exactly is distributed as "the tokenizer"? (a) A neural network. (b) An ordered list of merges. (c) A dictionary of words. (d) A compression codec.

5. Pre-tokenization exists primarily to: (a) Speed up training. (b) Prevent merges crossing meaningless boundaries. (c) Reduce vocabulary size. (d) Support Unicode.

6. `cat` and `cat` (with a leading space) are: (a) The same token. (b) Different tokens with different ids. (c) Invalid input. (d) Normalised to the same id.

7. Which content type typically costs the most tokens per byte? (a) English prose. (b) Minified JSON. (c) A base64 blob. (d) Python code.

8. Devanagari text costs many more tokens than English mainly because: (a) The language is more complex. (b) Its characters occupy more UTF-8 bytes and attract fewer learned merges. (c) Models refuse non-Latin script. (d) It is always translated first.

9. The token counting endpoint is: (a) Billed per call. (b) Free, with its own rate limits. (c) Only available on Opus. (d) Deprecated.

10. Token counts returned by the counting endpoint are: (a) Exact and guaranteed. (b) An estimate that may differ slightly. (c) Rounded to the nearest hundred. (d) Output tokens only.

11. Using `tiktoken` to count Claude tokens: (a) Is the documented approach. (b) Undercounts, reportedly by roughly 15–20%. (c) Overcounts safely. (d) Works for English only.

12. A team migrates to a newer model and its bill rises ~30% with identical traffic. The most likely cause is: (a) A price increase. (b) A different tokenizer producing more tokens. (c) A bug in their retries. (d) Rate limiting.

13. Cosine similarity of +1 between two embeddings means: (a) Identical text. (b) Identical direction in the vector space. (c) Identical length. (d) The vectors are perpendicular.

14. If embedding vectors are normalised to unit length, cosine similarity equals: (a) Euclidean distance. (b) The dot product. (c) The sum of components. (d) Nothing simpler.

15. A token embedding differs from a passage embedding in that the token embedding: (a) Is context-sensitive. (b) Is a static lookup, identical in every context. (c) Requires a forward pass. (d) Is always larger.

16. Anthropic's documented position on embedding models is that: (a) It offers `claude-embed-1`. (b) It offers none and points to third-party providers. (c) Embeddings are built into the Messages API. (d) Only Bedrock users get them.

17. Passing `input_type="document"` when embedding a user's search query will: (a) Raise a 400 error. (b) Silently degrade retrieval quality. (c) Have no effect. (d) Double the cost.

18. After switching to a different embedding model, the existing vector index must be: (a) Left alone. (b) Re-embedded entirely. (c) Merely re-normalised. (d) Sorted again.

19. Chunking should be budgeted in: (a) Characters. (b) Tokens. (c) Paragraphs. (d) Sentences.

20. Chunking 2,000 tokens at a 128-token budget with 24 tokens of overlap increases stored tokens by roughly: (a) 0%. (b) 5%. (c) 23%. (d) 100%.

6.20 Answers and Explanations

1 — (b). The embedding and unembedding matrices have one row per vocabulary entry, so the vocabulary must be fixed before training. (a) confuses runtime with training. (c) would require rebuilding the model per request. (d) confuses the vocabulary with the context window, which is a sequence-length limit, not a vocabulary limit.

2 — (b). Bottoming out at all 256 byte values makes the tokenizer total. (a) is exactly wrong — section 6.5 measured a 6.9x spread. (c) describes word-level tokenization, which BPE specifically is not. (d) is false: vocabularies differ by vendor, which is why question 11 exists.

3 — (b). Frequency is the entire selection criterion. (a) is a plausible-sounding distractor but length plays no part. (c) would be deterministic but would learn nothing. (d) inverts the algorithm and would produce a vocabulary of noise.

4 — (b). The ordered merge list is the tokenizer, because encoding means applying merges in learned order. (a) is false: there is no gradient descent in a tokenizer. (c) describes word-level tokenization. (d) describes BPE's 1994 ancestry, not what is shipped.

5 — (b). Section 6.3 showed the naive failure and the fix. (a) is a real side benefit but not the purpose. (c) is wrong — pre-tokenization slightly reduces compression at a given vocabulary size. (d) is handled by byte-level encoding, not by pre-tokenization.

6 — (b). The pattern `?\p{L}+` attaches a leading space to the following word, so the two strings are different chunks and get different ids. (a) and (d) would require a normalisation step that deliberately does not exist. (c) is nonsense — both are valid.

7 — (c). Base64 measured at 1.02 bytes per token, essentially one token per character. (a) was the cheapest at 3.76. (b) and (d) are expensive but nowhere near base64.

8 — (b). Three-byte UTF-8 characters plus few learned merges gives one token per byte. (a) is not a mechanism. (c) is false — byte-level BPE never refuses input. (d) describes no real system.

9 — (b). Documented as free, with its own RPM limits independent of the Messages API. (a) contradicts the documentation. (c) is false; all active models support it. (d) is false.

10 — (b). The documentation explicitly calls the count an estimate that may differ by a small amount. (a) overstates the guarantee. (c) is invented. (d) is backwards — it returns input tokens.

11 — (b). Different vocabularies produce different counts, and the undercount is the reported failure mode. (a) is the opposite of the guidance. (c) is wrong in both direction and safety: undercounting is what actually happens, and it is the dangerous direction. (d) is false; the mismatch is not language-specific.

12 — (b). This is the chapter's opening incident, and the documented ~30% tokenizer change is the explanation. (a) is contradicted by the fact that nothing about pricing changed. (c) would show as increased request counts, not increased tokens per request. (d) reduces throughput; it does not increase spend.

13 — (b). Cosine measures orientation only. (a) is too strong — different texts can have near-identical directions. (c) is what cosine deliberately discards. (d) describes a value of 0.

14 — (b). With both magnitudes equal to 1, the denominator vanishes. (a) is a different metric, though it is monotonically related on normalised vectors. (c) and (d) are wrong.

15 — (b). A token embedding is a row lookup, identical for `bank` in every sentence; context-sensitivity is produced by the attention layers above it. (a) inverts the answer. (c) describes the passage embedding. (d) is often false — model width frequently exceeds embedding dimension.

16 — (b). The documentation states Anthropic does not offer its own embedding model and names a third-party provider. (a) is an invented model id. (c) is false. (d) is false.

17 — (b). The scheme is asymmetric by design, and the wrong `input_type` is accepted without complaint. (a) is the tempting answer but no error is raised, which is exactly what makes this bug survive

to production. (c) is false — it measurably changes the vector. (d) confuses correctness with billing.

18 — (b). Vectors from different models are not comparable; cosine similarity across them is meaningless. (a) is the failure mode. (c) does not make incomparable spaces comparable. (d) is irrelevant.

19 — (b). Every binding constraint — embedding input limit, context window, prompt budget — is measured in tokens. (a), (c) and (d) are proxies that vary by an order of magnitude against the real constraint.

20 — (c). The measured figure in section 6.8.1 was 1.23x, and the general formula is `1 / (1 - overlap_fraction)`, here `1 / (1 - 24/128) = 1.23`. (a) ignores overlap entirely. (b) understates it. (d) would require 50% overlap.

6.21 Ten Practical Exercises

1. Train `MiniBPE` on 10 KB of your own text at vocabulary sizes 300, 500 and 1,000, and tabulate bytes per token.
2. Print the first 50 merges learned on an English corpus and annotate which are prefixes, which are suffixes and which are whole words.
3. Modify `GPT2_PAT` to remove the digit rule, retrain, and find a number that now tokenizes inconsistently.
4. Write a function that, given any string, returns the token count and flags the substring with the worst bytes-per-token ratio.
5. Measure bytes per token for the ten most common languages in your product's traffic and produce a fairness-gap table.
6. Take a real JSON payload from your system, minify it, and measure the token saving.
7. Replace every UUID in a sample prompt with a short key and measure the token reduction.
8. Compute cosine similarity between five pairs of sentences you consider similar and five you consider unrelated; check whether the numbers separate cleanly, and if they do not, explain why.
9. Chunk a long document at three budgets and measure how often a known answer is split across a boundary.
10. Write a pre-flight check that refuses to send any request whose counted token total exceeds a configured budget, and log the counted value.

6.22 Five Challenge Labs

1. **Vocabulary transplant.** Train two tokenizers on different corpora at the same vocabulary size and quantify how much worse each performs on the other's corpus. Explain the result in terms of merge order.
2. **A multilingual tokenizer.** Assemble a corpus in five scripts, train a single tokenizer on it, and reduce the fairness gap from section 6.5 as far as you can. Report the gap before and after.
3. **A regex-free pre-tokenizer.** Replace `GPT2_PAT` with hand-written character-class logic that produces identical chunks on a 1 MB corpus. Prove equivalence by test.

4. **Cost-aware prompt assembly.** Build a function that fills a token budget with retrieved chunks by descending relevance, stopping before the budget is exceeded, and compare its cost variance against fixed top-k.
5. **Tokenizer drift detector.** Build a monitor that counts a fixed canary prompt daily against your production model id and alerts on any change. Explain what a change would mean and what you would do about it.

6.23 Interview Questions

1. What is a tokenizer, and why do models use subword units rather than words or characters?
2. Walk me through byte-pair encoding. What is actually stored when you save a tokenizer?
3. Why do real tokenizers apply a regular expression before BPE? What breaks without it?
4. Why do `cat` and `cat` have different token ids, and what does that mean for prompt design?
5. Our bill went up 30% after a model migration with no traffic change. What happened and how would you confirm it?
6. How would you correctly count tokens for a request containing a system prompt, three tools and an image?
7. Why does the same sentence cost more tokens in Hindi than in English, and what would you do about it commercially?
8. What is the difference between a token embedding and a passage embedding?
9. Why does cosine similarity ignore vector length, and when would that be the wrong choice?
10. We changed our embedding model and search quality collapsed. What is the most likely cause?

6.24 Summary

A tokenizer is a deterministic, reversible function from text to integers, fixed before training and unchangeable afterwards because the model's embedding and unembedding matrices are sized to its vocabulary. Subword tokenization won over both character-level and word-level schemes because it is the only design that keeps sequences short, keeps the vocabulary bounded and never fails on unfamiliar input. That last property is a double edge: byte-level tokenizers do not reject strange input, they merely charge more for it, silently and indefinitely.

Byte-pair encoding is simple enough to implement in an afternoon. Start from the 256 byte values, repeatedly merge the most frequent adjacent pair, and record the merges in order. That ordered list *is* the tokenizer, and encoding means replaying it. Left unconstrained, the algorithm merges across word boundaries and degenerates into a lookup table for its training corpus — we watched it produce `'the cat s'` as a single token. Every production tokenizer therefore pre-tokenizes with a regular expression first, which is where the rule that a leading space belongs to the following word comes from, and why a trailing space in your prompt is a real defect rather than a cosmetic one.

The consequences are commercial before they are technical. The same sentence cost between 1.7 and 6.9 times more tokens in nine other languages than in English under our English-trained tokenizer; production multilingual tokenizers narrow that gap substantially but never close it. Base64 and UUIDs

cost roughly one token per character. Pretty-printed JSON cost 16% more than the same data minified. And because token counts are a property of the model rather than of the text, migrating between model generations can change your entire cost base overnight — as the documented ~30% increase for the newer tokenizer demonstrates. The discipline that follows is short: count, never estimate; count against the model you will actually use; and re-count before you migrate.

An embedding is where a token id stops being arbitrary. The model looks up a learned vector, and those vectors arrange themselves so that direction encodes meaning — a property nobody designed and which falls out of the training objective. Cosine similarity compares direction while discarding length, and on the normalised vectors most providers return it reduces to a plain dot product. Our 64-dimensional embeddings built from nothing but co-occurrence counts found `decoder` as the nearest neighbour of `encoder` at 0.841 and `latency` as the nearest neighbour of `cost` at 0.798 — and also found function words nearest to `model`, an honest reminder that frequency artefacts masquerade as meaning unless you weight for them.

Finally, the distinction the exam cares about: a token embedding is a static row lookup, identical in every context, while a passage embedding is the output of a whole encoder and is context-sensitive throughout. Anthropic does not publish an embedding model, pointing instead to third-party providers, whose vectors arrive normalised and whose asymmetric `input_type` parameter silently degrades retrieval if you get it backwards. Chunking those passages is not text processing; it is budgeting against four simultaneous token limits, where a 24-token overlap on 128-token chunks costs a measured 23% more of everything. Tokens are the unit of cost, the unit of the limit and the unit of latency — and now they are also the unit you know how to measure.

KEY TAKEAWAYS

1. **The token is the unit of cost, context limit and latency**, and the mapping from text to tokens belongs to the model, not to your text.
2. **A tokenizer is an ordered list of merges**, learned by counting, not by gradient descent, and fixed before training begins.
3. **Byte-level BPE never fails on unfamiliar input** — it charges more for it. Unfamiliar text is expensive, not rejected.
4. **Pre-tokenization is what makes BPE usable.** Without it, merges cross word boundaries and the vocabulary degenerates.
5. **A leading space is part of the token.** Never end a prompt with a trailing space.
6. **Count tokens with the endpoint; never estimate.** It is free, and it is the only number that matches the invoice.
7. **Token counts are model-specific.** Re-measure before every migration; the tokenizer has changed between generations.
8. **Non-English text and structured or random data cost substantially more per unit of meaning** — and the worst-tokenized languages are often the lowest-quality ones too.
9. **Cosine similarity compares direction only**, and equals the dot product on the normalised vectors providers return.
10. **Chunking is token budgeting**, embedding models are not interchangeable without re-indexing, and Anthropic publishes no embedding model of its own.

6.25 Glossary

Term	Definition
Base64	A text encoding of binary data that tokenizes at roughly one token per character.
Byte-level BPE	BPE whose base alphabet is the 256 byte values, guaranteeing no out-of-vocabulary input.
Byte-pair encoding (BPE)	Building a vocabulary by repeatedly merging the most frequent adjacent pair.
Bytes per token	Compression measure; higher means cheaper text for a given tokenizer.
Chat template	The transformation of structured messages into one flat token sequence using special tokens.
Chunking	Splitting documents into retrievable units, budgeted in tokens.
Cosine similarity	The cosine of the angle between two vectors; equals the dot product when both are unit length.
Embedding matrix	The `vocabulary x d` table of learned vectors, one row per token id.
Fairness gap	Bytes per token in the worst-served language divided by the best-served; a cost-equity metric.
Merge list	The ordered record of BPE merges; the tokenizer's actual content.
Out-of-vocabulary (OOV)	Input a tokenizer cannot represent — impossible for byte-level schemes.
Overlap	Shared tokens between adjacent chunks; costs `1 / (1 – fraction)` in storage.
Passage embedding	One vector summarising a whole passage, produced by a full encoder pass.
Pointwise mutual information	A co-occurrence weighting that down-weights merely frequent pairs.
Pre-tokenization	A regex split applied before BPE, forbidding merges across boundaries.
Special token	A vocabulary entry marking structure, reserved so user text cannot produce it.
Token	One entry of the vocabulary: a byte, fragment, word, punctuation mark or whitespace run.
Token embedding	The static vector looked up for a token id, identical in every context.
Token id	The integer index of a token in the vocabulary.
Tokenizer	The deterministic, reversible function converting text to token ids and back.
Vocabulary	The fixed, finite set of tokens a model knows; fixed before training.

6.26 Further Reading

Primary and canonical sources.

- Sennrich, Haddow and Birch (2015), *Neural Machine Translation of Rare Words with Subword Units*. The paper that brought BPE to language modelling.
- Gage (1994), *A New Algorithm for Data Compression*. The original compression algorithm.
- Radford et al. (2019), *Language Models are Unsupervised Multitask Learners*. Source of the byte-level BPE scheme and the pre-tokenization pattern used here.
- Kudo (2018), *Subword Regularization*, and Kudo and Richardson (2018), *SentencePiece*. The main alternative family, including the unigram model.
- Mikolov et al. (2013), *Efficient Estimation of Word Representations in Vector Space*. The origin of vector arithmetic on embeddings.
- Levy and Goldberg (2014), *Neural Word Embedding as Implicit Matrix Factorization*. Why the PPMI-plus-SVD recipe used in section 6.7 relates to neural embeddings.
- Claude Platform documentation, *Token counting* — the authoritative source for the counting endpoint, its rate limits and the tokenizer change between model generations.
- Claude Platform documentation, *Embeddings* and *Multilingual support* — the documented position on embedding providers and the cross-lingual performance figures quoted in section 6.5.

Where to go next in this book.

Chapter 7 completes the account of how a model comes to exist: pre-training, fine-tuning, RLHF, RLAIF and Constitutional AI — and picks up Lab 6.1's lesson that a statistical objective optimises what you measured rather than what you meant. Chapter 8 closes Part 1 with evaluation, hallucination and safety. The threads opened here run throughout: Chapter 17 on streaming and partial characters, Chapter 19 on pricing and quotas, Chapter 20 on the trailing-space defect and prompt design, Chapter 23 on the token cost of JSON contracts, Chapter 30 on prompt caching's 512-token minimum, Chapter 53 on chunking and retrieval in full, and Chapter 55 on attention dilution — the reason fewer tokens buys quality as well as money.